

Exception Handling - Complete Notes

Simple revision notes written in the same style we used in chat: why - how - when - interview.

1. Why Exception Handling exists

Java stops normal execution when something unexpected happens. Without exception handling, the program would crash immediately and the programmer would not get a chance to react. Exception handling was created so we can handle problems in a controlled way instead of letting the whole program die.

Simple idea
Problem happens
Java creates an exception object
Programmer can catch it and decide what to do
If nobody handles it, the program terminates

2. What is an Exception?

An exception is an object that represents an abnormal situation during program execution. It has a type, a message, and a stack trace. Example: division by zero creates `ArithmeticException`. Missing file creates `FileNotFoundException`.

Remember
Exception is not just a message.
It is a real Java object.
That object can be caught, inspected, and printed.

3. Exception hierarchy

Object
Throwable
/ \
Error Exception
RuntimeException
Arithmetic NullPointerException ArrayIndexOutOfBoundsException

Throwable is the parent of everything that can be thrown. Exception is for normal problems that applications may recover from. Error is for serious JVM problems and is usually not meant to be handled in normal application code.

4. Checked vs Unchecked Exceptions

Checked	Unchecked
Compiler checks them	Compiler does not force handling
Must use try-catch or throws	Optional to handle
Examples: IOException, SQLException	Examples: ArithmeticException, NullPointerException
Usually expected problems	Usually programming mistakes

Easy way to think: if the problem can happen even when your code is correct, it is usually treated like a checked exception. If the problem happens because of a bug in your logic, it is usually unchecked.

5. try, catch, finally

try means risky code. catch means handle the problem. finally means do cleanup work no matter what happens.

Flow
try - code that may fail
catch - handles the exception
finally - runs for cleanup

Example: if you open a file, catch can handle FileNotFoundException and finally can close the file or release resources.

Important rule: catch handles the problem. finally does not handle it; it cleans up after the handling or after the failure.

6. How try-catch works step by step

1) Java enters the try block. 2) If risky code fails, Java creates an exception object. 3) Java looks for a matching catch block. 4) If a match is found, the catch block receives the exception object. 5) After catch finishes, the program continues after the try-catch block.

Example
try { 10 / 0; }
ArithmeticException is created
catch (ArithmeticException e) handles it
Program continues after catch

7. Order of catch blocks

Write child exception catch blocks before parent exception catch blocks. If you write `catch(Exception e)` first, the later child catch becomes unreachable and Java gives a compile-time error.

Correct order
<code>catch(ArithmeticException e)</code>
<code>catch(Exception e)</code>

Wrong order
<code>catch(Exception e)</code>
<code>catch(ArithmeticException e) // unreachable</code>

8. throw vs throws

throw	throws
Used inside the method	Used in the method signature
Actually creates and sends the exception object	Only declares that the method may throw it
Example: <code>throw new Exception("msg")</code>	Example: <code>void test() throws Exception</code>

One-line memory: `throw` = actual action, `throws` = declaration in method signature.

9. Why throws is written in the signature and throw inside the method

The method signature tells the caller that the method may produce an exception. The `throw` statement is written where the problem is detected. So `throws` warns, and `throw` acts.

Simple example
<code>void checkAge(int age) throws Exception {</code>
<code> if (age < 18) {</code>
<code> throw new Exception("Too young");</code>
<code> }</code>
<code>}</code>

10. Custom exceptions

A custom exception is your own exception class for a business rule. Example: `LowBalanceException`, `InvalidAgeException`, `DuplicateEmployeeException`. This makes the code clearer than throwing a very generic `Exception` for everything.

Example:

```

class LowBalanceException extends Exception {
public LowBalanceException(String msg) {
super(msg);
}
}

```

If a business rule fails, you can throw your custom exception and catch it separately. That makes the code easier to read and easier to maintain.

11. Exception object contents

The variable in catch, for example `catch(Exception e)`, points to the full exception object. That object contains the exception type, the message, and the stack trace. That is why methods like `getMessage()` and `printStackTrace()` work.

Example data inside an exception object
Type: <code>ArithmeticException</code>
Message: <code>/ by zero</code>
Stack trace: where the problem happened

12. Why Throwable exists

Throwable is the parent of both Exception and Error. Only objects that extend Throwable can be thrown with the throw statement. A normal object like Student cannot be thrown unless Student extends Throwable directly or through Exception.

Rule
throw can only throw Throwable objects.
Exception and Error both inherit from Throwable.

13. Exception propagation

If a method does not handle a checked exception, it can pass it upward using throws. This is called propagation. The exception moves from one method to the caller until someone handles it or the program ends.

Chain
Method A throws
Caller B handles or throws again
Caller C handles or throws again

14. finally and final

finally is a block that runs for cleanup. final is a keyword used to make variables, methods, or classes fixed. They sound similar but they do completely different jobs.

final
final variable - cannot be reassigned
final method - cannot be overridden
final class - cannot be inherited

15. try-with-resources

When you work with files, streams, or database resources, try-with-resources is cleaner than manual cleanup. Java automatically closes resources that implement `AutoCloseable`.

Example
try (FileReader reader = new FileReader("data.txt")) {
// use reader
}
reader closes automatically

This is often preferred over writing a long finally block for closing resources.

16. Common interview questions

Why do we need exception handling? What is the difference between checked and unchecked exceptions? What is the difference between throw and throws? Why must child catch blocks come before parent catch blocks? Why do we use finally? Why create custom exceptions? What is Throwable?

17. Short revision sheet

Topic	Meaning
Exception	Abnormal runtime object
Checked	Compiler forces handling or declaration
Unchecked	Compiler does not force handling
try	Risky code
catch	Handle the exception
finally	Cleanup code
throw	Actually throw exception object
throws	Declare possible exception in signature
Custom exception	Your own business-rule exception
Throwable	Parent of Exception and Error